GOSIM

GOSIM Shenzhen 2026

October 16-17, 2026 Shenzhen, China



GOSIM

GOSIM Shenzhen 2026

GOSIM SHENZHEN 2026 · DORA WORKSHOP

面向机器人示范数据采集的 DORA-rs 遥操作闭环系统

设计与实现 · 四个关键设计决策与踩过的坑

孙玉杰　道瑞（北京）智能科技有限公司 · 研发工程师



决定模型上限的不是架构，是数据；决定数据上限的，是采集链路

100ms

手眼延迟的容忍上限

这是人在做精细操作时的感知阈值，不是机器人硬件的指标。

延迟：操作者会先「变笨」

画面到从臂响应一慢，人就会不自觉放慢动作、反复微调，轨迹里塞满犹豫和抖动，而这些噪声会被策略原封不动地学过去。

错位：学到的是错误因果

图像帧和关节状态来自不同设备、不同频率，时间戳不同源，动作和画面对不上——策略学到的不是「看到什么才做什么」。

不可复现：质量只能靠感觉

Episode 采完既不能回放也不能验证，数据集好坏只能靠「看起来还行」来判断，出了问题无从定位。

这三个问题一个都不发生在模型里，全发生在采集链路上——所以我把重心放在采集链路上。

我要解决的，只有这一件事

一句话定位

构建从人手示范到数据集落地的可复现闭环

硬件为 SO-101 主从臂与多路相机，基于 DORA-rs 从零构建采集链路，直至可稳定产出可用数据集。

今天不讲

DORA 基础概念

模型结构与训练细节

行业综述与路线之争

01 控制回路为什么放进 DORA

共享内存零拷贝，频率与延迟直接决定手感。

02 多模态时间对齐怎么做

图像帧与关节状态统一时钟，对齐误差决定数据能不能用。

03 采集即数据集

直接落 LeRobot 格式，配合 record / replay 可回放可验证。

04 怎么接住单点故障

节点崩了不能让整条链路静默停摆——用 dora 1.0 的容错机制兜住。

每个决策我都会同时给出代价——只讲好处的架构分享是不完整的。

系统全貌：一条数据流讲完



虚线为状态回传路径：双视角画面与关节状态实时回到操作者，构成操作闭环。

一次采集 → **可回放 · 可复现 · 可直接训练** · 整条链路仅一张 YAML 描述，节点可单独替换

Leader 主臂节点

读取关节角，作为示范输入源。

相机节点 ×2

环境相机全局视角，腕部相机操作特写。

映射与对齐节点

关节指令映射，多模态统一时钟。

Follower 从臂节点

实时跟随，回读真实关节状态。

录制节点

同步落盘关节序列与视频。

决策一：控制回路为什么放进 DORA

省掉的是开销项，不是靠参数调优

跨进程序列化与拷贝

常见做法：关节数组先序列化成字节，再整包拷给对端

DORA：共享内存传 Arrow 数组引用，不序列化、不过网络栈

采集端与执行端两套链路

常见做法：控制走一套，录数据另开一套，两边各自采样

DORA：录制节点挂在同一条数据流上，录到的就是真正下发执行的那一份

多套中间件拼接

常见做法：控制用 A、录制用 B，两套生命周期各自管理、各自对齐

DORA：控制与录制是同一张数据流图上的节点，一次启动、一套时钟

我们怎么量 · 三个量分开看，不混用

频率	dora topic hz <topic>，直接读发布频率，跑满 10 秒取稳定段
延迟	主臂发出打 t1、从臂收到打 t2，逐帧求差取 p50
口径	频率不等于延迟；parquet 里的落盘帧率也不等于指令频率

本机环境

SO-101 主从臂 ×2 · 相机 ×2

问题

主从臂之间的通信频率与延迟直接决定手感。频率低、延迟高时，操作者不敢做快动作，采出来的轨迹全是慢速和小幅抖动。

我的选择

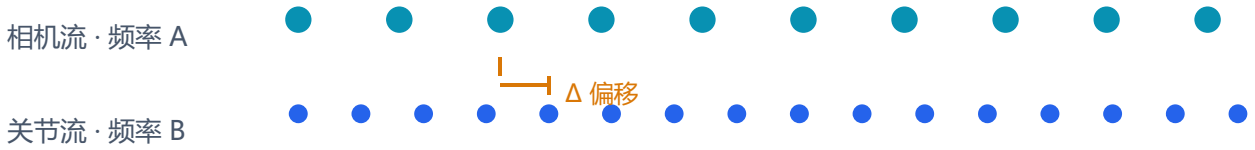
把主从臂放进同一条 DORA 数据流，用共享内存零拷贝承载，避掉跨进程序列化与拷贝开销。

代价

节点全塞进一条数据流，故障面也跟着集中——**默认配置下，一个节点挂掉，整条链路就静默停摆**。

决策二：多模态时间对齐怎么做

对齐前 · 两套时钟，各写各的时间戳



统一时钟基准 · 关节状态按最近邻对齐到图像帧

对齐后 · 每一帧都能找到唯一对应的动作



对齐前：两条流各写各的时间戳 · 对齐后：任一图像帧都能索引到唯一关节状态

问题

相机和关节是两个设备、两条时钟，频率也不相同，落盘时各写各的时间戳。动作和画面对不上，策略学到的就是错误因果。

我的选择

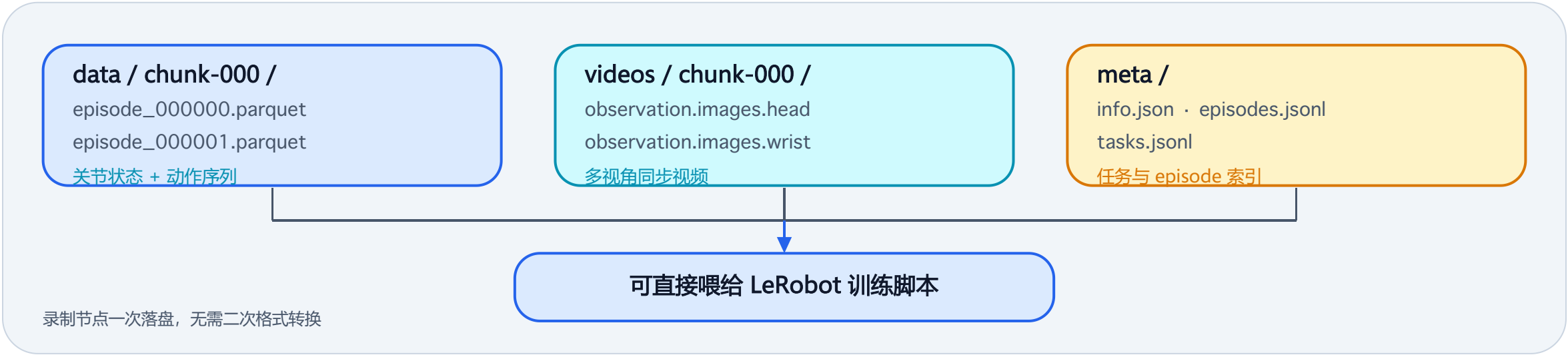
统一到同一时钟基准，再用最近邻把关节状态对齐到图像帧，保证每一帧都有唯一对应的动作标注。

代价

最近邻会引入量化误差，动作越快误差占比越大——所以采集时要限制单步关节跳变幅度。

对齐不是锦上添花，是采集质量的命门。这一项做不好，后面换什么模型都救不回来。

决策三：采集即数据集



采完即训

录制节点直接写 LeRobot 格式，不做二次转换，采集结束那一刻数据集就是可用的。

可回放进 CI

用 record / replay 原样重放 episode，回归测试拿录好的数据跑新逻辑，不必接真机。

代价要讲清楚

LeRobot 有它自己的字段约定，自定义字段一不小心就会破坏兼容性，需要单独校验。

决策四：怎么接住单点故障

● 采集到第 37 条 episode，相机节点崩了　　默认行为：整条链路静默停摆

dora 1.0 的四层防线 · 自上而下逐层兜底

restart_policy

崩溃即重启，指数退避，可设次数上限与窗口

health_check_timeout

接住「假死」：进程还在、不再通信，daemon 主动查杀

input_timeout + 断路器

上游断流，下游收到 InputClosed；数据一恢复自动重连

NodeRestarted 事件

上游换了新进程，下游收到通知，清掉旧实例的缓存状态

另：coordinator 可用 --store redb 落盘，coordinator 自身重启后各 daemon 自动重连、状态不丢

我的配置

```
health_check_interval: 2.0
nodes:
  - id: camera_wrist
    restart_policy: on-failure
    health_check_timeout: 10.0
    max_restarts: 5
    restart_delay: 1.0
    inputs:
      tick:
        input_timeout: 5.0
```

代价

自动重启不等于无损。控制回路节点重启的那几秒，从臂是失能的——必须有兜底位姿，并且把这段 episode 标为不可用，别让它混进训练集。

这四层都不是默认开启的。**不写就什么都不做**——这是我最想提醒的一点。



踩坑（上）：环境坑

会报错，
但报错指不到根因

这三个都发生在开跑之前。它们不隐蔽，但错误信息会指向完全错误的方向。



坑一：dora-rs 与 dora-cli 版本必须严格一致

差一个小版本数据流就起不来，而报出来的错完全指不到根因。装环境时第一件事就是对版本，别等报错再回头查。

坑二：相机接入顺序决定设备号

先插的是 /dev/video0。插错顺序拿到的就是错视角，而画面看起来完全正常——等训出来才发现，一整批数据作废。

坑三：daemon 异常退出会留下共享内存残留

上次跑崩留下的共享内存段和僵尸进程，会让下一次启动报出莫名其妙的错。先 destroy 再 start，能省掉大量无效排查。

共同点：三个都会当场报错，但报错指不到根因——排查时间全花在猜上。

踩坑（下）：数据坑

坑四：时间戳必须用同一时钟源

各设备各写各的时间戳，落盘后每一条都看起来正常，只有训练结果会告诉你不对——而这个反馈链长得足以浪费一整周。

坑五：标定文件没生效，肉眼看不出来

路径写错或没加载时，从臂照样动、照样跟手，只是关节零位偏了几度——所有轨迹系统性偏移，采集时完全无感。

坑六：丢帧与 episode 边界没人校验

相机标称 30Hz，实际只写进 20 帧，视频时长和标注条数对不上。落盘时必须做条数自检，否则 episode 静默损坏。

共同点：三个都不报错。它们的代价不是排查时间，是整批数据。

不报错，
但你以为成功了

这三个比环境坑贵得多。它们全程静默，等你从训练结果里发现，一整批数据已经废了。



GOSIM

GOSIM Shenzhen 2026

THANK YOU

采集链路值得被当作一等公民来设计

把链路做对，模型才有机会。

欢迎会后找我聊遥操作、数据采集，或者直接来提 issue。

